Unidad 4: Programación Orientada a Objetos, Interfaces Gráficas y Bases de Datos

FICHA TÉCNICA DE LA UNIDAD

Unidad trabajo	UT4. Programación Orientada a Objetos, Interfaces Gráficas y Bases de Datos
Resultados de Aprendizaje	RA3. Diseña y programa componentes lógicos utilizando el paradigma de orientación a objetos (POO) y gestionando excepciones.
Criterios de Evaluación	CE3.a: Asimilar los conceptos básicos de clase, instancia, objeto, atributos y constructores. CE3.b: Implementar mecanismos de encapsulamiento para proteger el acceso interno a las propiedades del objeto. CE3.c: Programar estructuras complejas utilizando herencia y polimorfismo. CE3.d: Diseñar y gestionar bloques de control de excepciones. CE3.e: Identificar y aplicar patrones de diseño de software básicos.
Tiempo	12 horas

ÍNDICE DE CONTENIDOS

1. Introducción a la POO

- 1.1. Conceptos Fundamentales y Analogías
- 1.2. Declaración de Clases e Instanciación en Python

2. Ciclo de Vida del Objeto y Métodos Avanzados

- 2.1. El Método Constructor __init__
- 2.2. Colecciones de Objetos (Listas de Instancias)
- 2.3. Encapsulamiento y Control de Acceso (Getters y Setters)

3. Relaciones entre Clases

- 3.1. Herencia Simple y la Función super()
- 3.2. Delegación / Composición
- 3.3. Herencia Múltiple

3.4. Atributos y Métodos Estáticos (de Clase)

3.5. Métodos Especiales (Mágicos / Dunder)

3.6. Polimorfismo e Inspección de Tipos

4. Interfaces Gráficas de Usuario (GUI) con Tkinter

4.1. El Ciclo de Eventos y Conceptos Base

4.2. Métodos de Disposición / Layout Managers

4.3. Ejemplo Práctico Integrador: Interfaz de Entrada de Datos

5. Conexión con Sistemas de Base de Datos (MySQL)

5.1. Componentes del Entorno de Conexión

5.2. Gestión Segura de Conexiones y Operaciones CRUD

1. Introducción a la POO

Hasta ahora se ha trabajado utilizando la programación estructurada tradicional, fundamentada en variables simples o complejas (como listas o cadenas de caracteres) de un tipo concreto y estructuras de control secuenciales, condicionales o iterativas. Sin embargo, para modelar de manera óptima elementos del mundo real que contienen datos de diferente naturaleza, los lenguajes de programación modernos implementan la Programación Orientada a Objetos (POO).

En el ecosistema de Python, casi todo es un objeto. Cada vez que declaramos una variable, no solo almacenamos un dato aislado, sino que instanciamos una clase que hereda métodos y herramientas predefinidas de forma interna. Por ejemplo, al transformar un texto a mayúsculas mediante nombre.upper(), invocamos un método nativo de la clase str; al emplear frutas.append(), alteramos el estado interno de un objeto de la clase list.

1.1. Conceptos Fundamentales y Analogías

Para asimilar la transición conceptual, se establece la siguiente correspondencia entre la programación tradicional y el paradigma orientado a objetos:

Concepto POO	Definición Técnica	Analogía / Ejemplo Tradicional
Clase	Representación abstracta, molde o modelo de un concepto que define sus características y comportamientos compartidos.	Tipo de dato (ej. str, list, int, o conceptos personalizados como "Servidor Web").
Objeto / Instancia	Manifestación concreta y real construida a partir del molde de una clase, con existencia propia en memoria.	Variable real (ej. "Hola", [1, 2, 3], o un servidor físico determinado).

Concepto POO	Definición Técnica	Analogía / Ejemplo Tradicional
Atributo / Propiedad	Variables internas asociadas a la clase o al objeto que definen sus rasgos, características o estado.	Datos del elemento (ej. el tamaño de una lista, el DNI de un usuario, la IP de un router).
Método	Bloques de código o funciones internas de una clase diseñadas para definir el comportamiento y operar sobre sus atributos.	Funciones vinculadas al objeto invocables mediante el operador punto (ej. .upper(), .append()).

1.2. Declaración de Clases e Instanciación en Python

En lugar de mantener variables independientes desvinculadas para representar un mismo elemento (por ejemplo, variables sueltas como nombreAna, edadAna, dniAna), la POO unifica los datos bajo una estructura común de tipo objeto. El parámetro fundamental en los métodos de Python es self, una palabra reservada que por convención hace referencia directa a la propia instancia sobre la cual se está ejecutando el bloque de código, permitiendo manipular sus atributos internos.

```
# Ejemplo de declaración de clase base e instanciación básica
class Persona:
nombre = ""
apellidos = ""
dni = ""
edad = 0
peso = 0
altura = 0
def cambiaPeso(self, nuevoPeso):
# self permite acceder y modificar los atributos de la instancia
actual
self.weight = nuevoPeso
# Instanciación (creación de un objeto concreto a partir de la
clase)
pedro = Persona()
pedro.nombre = "Pedro"
pedro.apellidos = "Ruiz Serrano"
pedro.cambiaPeso(87)
if pedro.edad < 18:
print("Es menor de edad")
```

2. Ciclo de Vida del Objeto y Métodos Avanzados

2.1. El Método Constructor __init__

Los atributos de un objeto no se reservan formalmente en memoria con un estado personalizado hasta que no se inicializan de forma explícita. Para automatizar este proceso y garantizar que toda instancia cuente con valores de arranque válidos desde su nacimiento, se utiliza el método especial constructor __init__.

```
class Persona:
def __init__(self, nombre, apellidos, dni, edad, peso, altura):
# Inicialización de los atributos de instancia mediante el parámetro
self
self.nombre = nombre
self.apellidos = apellidos
self.dni = dni
self.edad = edad
self.peso = peso
self.altura = altura
# Creación de la instancia pasando los argumentos requeridos por el
constructor
pedro = Persona('Pedro', 'Ruiz Serrano', '87654321G', 21, 78,
1.67)
```

2.2. Colecciones de Objetos (Listas de Instancias)

Las instancias de clases personalizadas pueden almacenarse y manipularse dentro de estructuras de datos estándar como las listas, simulando el comportamiento organizativo de los registros de una base de datos relacional y permitiendo aplicar métodos nativos como append(), pop() o insert().

```
personas = []
personas.append(Persona('Pedro', 'Ruiz Serrano', '87654321G', 21, 78,
1.67))
personas.append(Persona('Ana', 'Cobo Sánchez', '12345678T', 26, 65,
1.77))
personas.append(Persona('Juan Luis', 'Rodríguez Pérez', '874534330',
45, 67, 1.87))
# Acceso al atributo de un objeto específico dentro de la lista
# Imprime: "Juan Luis"
print(personas[2].nombre)
```

2.3. Encapsulamiento y Control de Acceso (Getters y Setters)

El principio de encapsulación busca restringir el acceso directo a los datos internos de un objeto para preservar su integridad y validar las modificaciones que se realicen sobre ellos. En Python, los atributos privados se definen anteponiendo un doble guion bajo (__), impidiendo que el exterior los lea o modifique de forma directa (provocando un error de tipo AttributeError). Para gestionarlos con elegancia sin perder la sintaxis limpia de atributos, se emplean los decoradores @property (Getters) y @<atributo>.setter (Setters).

```
class Circulo:
def __init__(self, radio):
self.radio = radio # Invoca automáticamente al setter para su
validación
@property
def radio(self):
""" Método Getter: Expone el atributo protegido como una propiedad de
lectura """
print("Accediendo al valor del radio")
return self.__radio
@radio.setter
def radio(self, valor):
""" Método Setter: Intercepta la asignación para validar el dato de
entrada """
if valor >= 0:
self.__radio = valor
else:
print("Error: El radio debe ser un valor positivo.")
self.__radio = 0
# Prueba de ejecución en consola interactiva
c1 = Circulo(3) # Inicializa
print(c1.radio) # Llama al getter. Imprime 3
c1.radio = -1 # Llama al setter. Muestra el mensaje de error y lo
fija en 0
```

3. Relaciones entre Clases

3.1. Herencia Simple y la Función super()

La herencia es un pilar de la POO que permite construir una clase nueva (clase derivada o clase hija) a partir de una existente (clase base o clase padre), heredando íntegramente sus atributos y métodos. La función integrada super() proporciona una referencia directa a la clase base, posibilitando la reutilización o extensión de sus métodos mediante la técnica de sobreescritura.

```
import math
class Punto:
def __init__(self, x=0, y=0):
self.x = x
self.y = y
def mostrar(self):
return f"{self.x}:{self.y}"
def distancia(self, otro):
dx = self.x - otro.x
dy = self.y - otro.y
```

```
return math.sqrt(dx*dx + dy*dy)
class Punto3D(Punto):
""" Clase derivada que extiende a la clase Punto añadiendo una
coordenada Z """
def __init__(self, x=0, y=0, z=0):
super().__init__(x, y) # Invoca al constructor de la clase padre
(Punto)
self.z = z
def mostrar(self):
# Reutiliza el método de la clase base y añade la dimensión Z
return f"{super().mostrar()}:{self.z}"
```

3.2. Delegación / Composición

Ocurre cuando una clase no hereda de otra, sino que incorpora una o más instancias de otras clases como atributos internos, delegando en ellas parte de su comportamiento lógico o responsabilidades de cálculo.

```
class CirculoCentro:
def __init__(self, centro, radio):
# 'centro' se espera que sea una instancia previamente creada de la
clase Punto
self.centro = centro
self.radio = radio
def mostrar(self):
# Delega en el método mostrar de la instancia de Punto almacenada en
self.centro
return f"Centro: {self.centro.mostrar()} - Radio: {self.radio}"
```

3.3. Herencia Múltiple

Python permite de forma nativa que una clase derivada provenga de múltiples clases superiores. En este escenario, es crítico estructurar y nombrar los métodos con claridad para evitar colisiones de nombres o ambigüedades en la resolución.

```
class Telefono:
def __init__(self, numero):
self.numero = numero
def telefonear(self): print("Llamando...")
class Camara:
def __init__(self, mpx):
self.mpx = mpx
def fotografiar(self): print("Fotografiando...")
class Reproductor:
def __init__(self, capacidad):
```

```
self.capacidad = capacidad
class Movil(Telefono, Camara, Reproductor):
""" Clase que hereda simultáneamente de tres superclases """
def __init__(self, numero, mpx, capacidad):
Telefono.__init__(self, numero)
Camara.__init__(self, mpx)
Reproductor.__init__(self, capacidad)
```

3.4. Atributos y Métodos Estáticos (de Clase)

Existen elementos que no pertenecen de forma individual a un objeto concreto, sino que son comunes a toda la clase:

- **Atributos de Clase:** Variables definidas directamente dentro de la clase pero fuera de sus métodos. Son estáticas y compartidas por todas las instancias.
- **Métodos Estáticos (@staticmethod):** Funciones englobadas en la clase por coherencia semántica u organización conceptual, pero que no requieren leer ni modificar datos específicos del objeto (no reciben self).

```
class Calculadora:
def __init__(self, modelo):
self.modelo = modelo
@staticmethod
def sumar(x, y):
""" Operación puramente lógica que no depende de atributos de
instancia """
return x + y
```

3.5. Métodos Especiales (Mágicos / Dunder)

Permiten sobreescribir comportamientos estándar de los operadores internos de Python (print, comparaciones, operaciones aritméticas):

Método Mágico	Acción que intercepta / Propósito
__str__(self)	Devuelve una representación de texto "informal" del objeto. Es el bloque que ejecuta print().
__repr__(self)	Devuelve una representación "formal" y unívoca del objeto (idealmente válida para ser procesada por eval()).
__eq__(self, otro)	Sobreescribe el operador de igualdad (==). Si no se define, Python compara las direcciones de memoria.
__add__(self, otro) / __sub__	Permiten definir el comportamiento lógico al aplicar los operadores matemáticos de suma (+) y resta (-).

3.6. Polimorfismo e Inspección de Tipos

El polimorfismo es la propiedad por la cual una misma llamada a un método puede producir efectos o respuestas diferentes según la naturaleza de la clase del objeto que lo invoque en tiempo de ejecución (aprovechando el tipado dinámico de Python). Para la comprobación de tipos y jerarquías en caliente, el lenguaje cuenta con herramientas integradas:

- isinstance(objeto, Clase): Determina si una instancia concreta pertenece a una clase específica o a su árbol jerárquico.
- issubclass(Subclase, Superclase): Verifica si una clase deriva directamente de otra de mayor jerarquía.

4. Interfaces Gráficas de Usuario (GUI) con Tkinter

Una Interfaz Gráfica de Usuario (GUI) es un entorno visual que abstrae al usuario de la interacción exclusiva por consola o línea de comandos, incorporando ventanas, menús, campos de texto y botones interactivos. En Python, la biblioteca estándar y nativa para este propósito es **Tkinter**, basada en el motor gráfico clásico Tk.

4.1. El Ciclo de Eventos y Conceptos Base

A diferencia de los scripts secuenciales, una aplicación de escritorio trabaja bajo un enfoque de **Gestión de Eventos**. El framework dibuja los componentes y cede el control a un bucle infinito de escucha activa:

- **Ventana Principal (Tk()):** El contenedor primario o marco estructural que sostiene el entorno de la aplicación.
- **Widgets:** Componentes visuales interactivos. Los fundamentales son: Label (etiquetas de texto/imágenes), Button (ejecutores de acciones), Entry (cajas de entrada de una sola línea) y Text (áreas de texto multilínea).
- **Bucle Principal (mainloop()):** El motor síncrono que bloquea la finalización del script, manteniendo la ventana abierta en espera de que el usuario haga clic, escriba o interactúe.

Regla de Oro en el Desarrollo: Nunca se debe guardar o nombrar un script propio bajo el nombre exacto de tkinter.py. De hacerlo, el sistema entrará en un fallo de importación circular (*Circular Import*) al intentar invocarse a sí mismo.

4.2. Métodos de Disposición / Layout Managers

Tkinter cuenta con tres mecanismos para organizar y distribuir los elementos espaciales dentro del lienzo de la ventana:

1. .pack(): Organiza y apila los elementos en bloques verticales u horizontales de forma automática. Es ágil y directo, pero poco flexible para composiciones complejas.

2. .grid(): Distribuye los componentes utilizando una estructura matricial invisible de filas (row) y columnas (column), similar a una hoja de cálculo. Es el estándar profesional para formularios limpios.

3. .place(): Posiciona los widgets asignando coordenadas de píxeles absolutas exactas (X e Y). Ofrece control total pero penaliza la adaptabilidad y el redimensionamiento de las ventanas.

4.3. Ejemplo Práctico Integrador: Interfaz de Entrada de Datos

El siguiente bloque muestra la forma idónea de programar una ventana interactiva, utilizando el método .grid() para asegurar un diseño limpio y enlazando eventos a funciones (callbacks) utilizando el parámetro command.

```
import tkinter as tk
def procesarSuma():
# .get() extrae la cadena del Entry; se castea a int para operar
aritméticamente
resultado = int(entrada_n1.get()) + int(entrada_n2.get())
# .config() permite actualizar en caliente cualquier propiedad del
widget visual
etiqueta_resultado.config(text=f"Resultado: {resultado}")
# Inicialización del contenedor principal
app = tk.Tk()
app.title("Calculadora Escolar ASIR")
app.geometry("280x160")
# Fila 0: Configuración de primer elemento
tk.Label(app, text="Número 1:").grid(row=0, column=0, padx=5,
pady=5)
entrada_n1 = tk.Entry(app)
entrada_n1.grid(row=0, column=1, padx=5, pady=5)
# Fila 1: Configuración de segundo elemento
tk.Label(app, text="Número 2:").grid(row=1, column=0, padx=5,
pady=5)
entrada_n2 = tk.Entry(app)
entrada_n2.grid(row=1, column=1, padx=5, pady=5)
# Fila 2: Botón de acción con Command (Ojo: se pasa la función sin
paréntesis)
boton_operar = tk.Button(app, text="Calcular Suma",
command=procesarSuma)
boton_operar.grid(row=2, column=0, columnspan=2, pady=10)
# Fila 3: Área destinada a proyectar la salida
etiqueta_resultado = tk.Label(app, text="Resultado: 0",
font=("Arial", 10, "bold"))
etiqueta_resultado.grid(row=3, column=0, columnspan=2, pady=5)
```

```
# Lanzamiento del bucle de escucha activa
app.mainloop()
```

5. Conexión con Sistemas de Base de Datos (MySQL)

Para dotar a los objetos de persistencia real en entornos de producción, es imperativo interconectarlos con Sistemas de Gestión de Bases de Datos Relacionales (RDBMS) remotos, como MySQL o MariaDB. Para este fin, se requiere el conector oficial de la API de base de datos de Python.

5.1. Componentes del Entorno de Conexión

- **Driver (Conector):** Componente de software encargado de traducir las llamadas y tipos de datos de Python al protocolo de red nativo del servidor de BD. Instalación: pip install mysql-connector-python.
- **Conexión (Connection Link):** Sesión lógica de comunicación activa establecida de forma segura entre el script cliente y la instancia del motor de base de datos.
- **Cursor:** Objeto operativo intermedio que actúa como puntero del sistema para despachar consultas SQL hacia el servidor, procesar sentencias y recuperar de forma ordenada las filas resultantes en memoria.

5.2. Gestión Segura de Conexiones y Operaciones CRUD

Es un principio crítico de administración e infraestructura de software asegurar que las conexiones se abran únicamente cuando sea necesario y se cierren siempre (incluso si ocurren fallos inesperados en el proceso de consulta), utilizando bloques estructurados de control de excepciones try-except-finally.

Excepción del Conector	Causa de Origen / Cuándo Capturarla
ProgrammingError	Errores críticos de sintaxis en la sentencia SQL escrita, llamadas a tablas que no existen o credenciales con privilegios insuficientes en el motor.
IntegrityError	Violaciones de las reglas de consistencia de datos de la BD (ej. duplicar una clave primaria, insertar nulos o violar restricciones de clave foránea).
InterfaceError	Fallos de enlace o comunicación física entre el script ejecutor de Python y el componente interno del Driver (ej. intentar usar una conexión ya cerrada).

```
import mysql.connector
from mysql.connector import Error, errorcode
try:
# Intento de establecimiento del canal de comunicación relacional
```

```
conexion = mysql.connector.connect(
host='IP_SERVIDOR_REMOTO',
user='USUARIO_ASIR',
password='PASSWORD_SEGURO',
database='EMPRESA_BD'
)
if conexion.is_connected():
print("Conexión establecida con éxito.")
cursor = conexion.cursor()
#
========================================================================
# 1. OPERACIÓN DE LECTURA SEGURA (Prevención estricta de Inyección
SQL)
#
========================================================================
# Regla de Oro: NUNCA concatenar variables o usar f-strings en
cadenas SQL.
# Se debe utilizar obligatoriamente marcadores de posición '%s' y
pasar tuplas.
query = "SELECT id, nombre, email FROM usuarios WHERE nombre =
%s"
parametro_busqueda = ("Alex",) # Las tuplas unitarias llevan coma
obligatoria
cursor.execute(query, parametro_busqueda)
resultados = cursor.fetchall() # fetchall() extrae todas las filas en
formato lista de tuplas
for fila in resultados:
print(f"ID: {fila[0]} | Nombre: {fila[1]} | Email: {fila[2]}")
#
========================================================================
# 2. OPERACIÓN DE ESCRITURA CON TRANSACCIONALIDAD
(INSERT/UPDATE/DELETE)
#
========================================================================
insert_sql = "INSERT INTO usuarios (nombre, email) VALUES (%s,
%s)"
datos_registro = ("Carla", "carla@empresa.com")
cursor.execute(insert_sql, datos_registro)
# Confirmación física y persistente de los cambios en el disco duro
del servidor
conexion.commit()
print(f"Registro asentado correctamente. ID asignado:
{cursor.lastrowid}")
except mysql.connector.Error as err:
# Manejo granular de códigos de error nativos del motor MySQL
if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
```

```
print("Error: Credenciales de acceso (usuario/password)
denegadas.")
elif err.errno == errorcode.ER_BAD_DB_ERROR:
print("Error: La base de datos especificada no existe en el
servidor.")
else:
print(f"Error operativo detectado: {err}")
# En caso de cualquier fallo en la escritura, revertimos los cambios
pendientes en la transacción
if 'conexion' in locals() and conexion.is_connected():
conexion.rollback()
finally:
# El bloque 'finally' garantiza la liberación del recurso de red pase
lo que pase
if 'conexion' in locals() and conexion.is_connected():
cursor.close()
conexion.close()
print("Recursos liberados. Conexión cerrada de forma limpia.")
```